Accenture
DevOps & Cloud
Interview Prep

14 Real Questions · Structured Answers · Interview-Ready

AWS · Kubernetes · Terraform · GCP · Networking · IAM

14	7	100+
Questions	Topic Areas	Bullet Points

Prepared for Accenture Interview Success

QUESTION 01 · Accenture DevOps Interview

What is the difference between Private Subnets and Public Subnets, and how are they used?

[AWS VPC] [Networking]

CORE DIFFERENCE

Subnet visibility and routing determine whether resources can communicate directly with the internet.

PUBLIC SUBNET

- **Route Table:** Has a route to an Internet Gateway (IGW)
- Resources assigned **public IP addresses** — reachable from the internet
- **Use cases:** ALB/NLB, NAT Gateway, Bastion hosts, static websites on EC2

PRIVATE SUBNET

- **Route Table:** No direct IGW route — no inbound internet access
- Outbound traffic routed through a **NAT Gateway** (placed in public subnet)
- **Use cases:** Application servers, RDS databases, EKS worker nodes, Lambda in VPC

ARCHITECTURE PATTERN

- Public Subnet → ALB / NAT GW / Bastion
- Private Subnet → EC2 App Servers → talk to RDS in private subnet
- Private Subnet → outbound internet via NAT GW (e.g., pull Docker images)

■ *Databases should ALWAYS be in private subnets — never expose port 5432/3306 to the internet.*

QUESTION 02 · Accenture DevOps Interview

What are the different types of Kubernetes Clusters?

[Kubernetes] [Architecture]

BY MANAGEMENT MODEL

- **Self-Managed (kubeadm):** You provision control plane + worker nodes manually. Full control, full responsibility.
- **Managed (Cloud-Provider):** Control plane managed by cloud. Only manage worker nodes.
  - → AWS EKS, GKE (GCP), AKS (Azure)
- **Local/Dev Clusters:** For local testing only — not production.
  - → Minikube, Kind (K8s in Docker), k3d, Docker Desktop K8s

BY NODE COMPUTE TYPE

- **EC2-based (standard):** You manage node groups, AMIs, instance types
- **Fargate (serverless):** AWS manages underlying nodes. No EC2 to patch. Used with EKS.
- **Spot/Preemptible:** Cost-optimized but interruptible — used with Cluster Autoscaler

BY NETWORK TOPOLOGY

- **Single-cluster:** All workloads in one cluster — simpler but single point of failure
- **Multi-cluster:** Separate clusters per env (dev/staging/prod) or region — higher resilience
- **Federation:** Centrally manage multiple clusters (used in large enterprises)

■ *In your Hisan Labs internship, you used AWS EKS with Fargate — highlight this as managed + serverless.*

QUESTION 03 · Accenture DevOps Interview

What GCP Kubernetes resources have you provisioned?

[GCP] [GKE] [Honesty Flag]

HONEST FRAMING FOR THIS QUESTION

Your hands-on experience is on **AWS EKS, not GKE**. Do not fabricate GCP experience. Use this framework:

RECOMMENDED ANSWER STRATEGY

- **Be direct:** "My Kubernetes experience is on AWS EKS. I have not provisioned GKE clusters professionally.
- **Bridge to AWS:** Describe what you provisioned on EKS — Fargate profiles, node groups, IAM OIDC/IRSA, Helm charts via Argo CD
- **Show transferability:** "GKE and EKS share the same core K8s API — I can ramp up on GKE-specific tooling quickly."

IF ASKED WHAT GKE RESOURCES LOOK LIKE (CONCEPTUAL)

- **GKE Standard:** Zonal or regional cluster, node pools, Workload Identity for IAM
- **GKE Autopilot:** Fully managed nodes (like EKS Fargate) — no node pool management
- **Workload Identity:** GCP equivalent of AWS IRSA — binds K8s SA to GCP Service Account
- **Anthos:** Google's multi-cluster/hybrid management platform

■ *Interviewers respect honesty + transferability far more than fabricated GCP experience.*

QUESTION 04 · Accenture DevOps Interview

What does a Terraform module look like, and what files should it contain?

[Terraform] [IaC]

MODULE STRUCTURE

```
my-module/
■■■ main.tf # Core resources
■■■ variables.tf # Input variable declarations
■■■ outputs.tf # Output value declarations
■■■ versions.tf # Required providers & Terraform version
■■■ README.md # Module documentation
```

FILE RESPONSIBILITIES

- **main.tf:** Defines resources (aws_instance, aws_vpc, etc.). Core logic lives here.
- **variables.tf:** Declares inputs — name, type, default, description, validation rules.
- **outputs.tf:** Exposes values to calling module — e.g., VPC ID, subnet IDs, ARNs.
- **versions.tf:** Pins required_providers and minimum Terraform version. Prevents drift.

CALLING A MODULE

```
module "vpc" {
source = "./modules/vpc"
region = var.aws_region
cidr = "10.0.0.0/16"
}
```

BEST PRACTICES

- Keep modules **single-responsibility** — one module per logical component (vpc, eks, rds)
- Always version-lock modules from Terraform Registry or Git tags
- Use **terraform-docs** to auto-generate README from variable/output descriptions

■ *In your Hisan Labs work, mention modular Terraform for VPC, EKS, and IAM — it shows production discipline.*

QUESTION 05 · Accenture DevOps Interview

What is a NAT Gateway?

[AWS] [Networking]

DEFINITION

A **NAT Gateway (Network Address Translation Gateway)** allows resources in a private subnet to initiate outbound connections to the internet while **blocking inbound connections** from the internet.

HOW IT WORKS

- Placed in a **public subnet** with an Elastic IP attached
- Private subnet route table: 0.0.0.0/0 → NAT Gateway
- NAT GW translates private IP → its own public Elastic IP for outbound traffic
- Internet sees only the NAT GW's Elastic IP — private IPs are never exposed

COMMON USE CASES

- EC2 in private subnet pulling OS updates (yum/apt)
- EKS worker nodes pulling container images from Docker Hub / ECR
- Lambda in VPC calling external APIs
- RDS calling AWS Secrets Manager endpoint (if no VPC Endpoint)

NAT GATEWAY VS NAT INSTANCE

- **NAT Gateway:** AWS-managed, highly available per AZ, no patching required — preferred
- **NAT Instance:** EC2-based, manual HA setup, cheaper but operational overhead — legacy

■ *For HA, deploy one NAT Gateway per AZ. Cross-AZ NAT traffic incurs data transfer charges.*

QUESTION 06 · Accenture DevOps Interview

What is a Static Pod, and how is it provisioned and handled?

[Kubernetes] [Control Plane]

WHAT IS A STATIC POD?

A **Static Pod** is a pod managed directly by the kubelet on a specific node — **not by the API server or kube-scheduler**. The kubelet watches a directory for pod manifests and creates/restarts them automatically.

HOW IT'S PROVISIONED

- Place a pod manifest YAML in the kubelet's staticPodPath directory
- Default path: /etc/kubernetes/manifests/
- Kubelet polls this directory — creates pod when file appears, deletes when file is removed

```
# Example: kubelet config
staticPodPath: /etc/kubernetes/manifests
```

HOW THEY'RE HANDLED

- Kubelet creates a **mirror pod** on the API server (read-only view — can't kubectl delete it)
- If the static pod crashes, kubelet **auto-restarts** it without scheduling logic
- No ReplicaSet or Deployment controls them

REAL-WORLD USAGE

- **Control plane components** in kubeadm clusters are static pods:
  - kube-apiserver, kube-controller-manager, kube-scheduler, etcd
- This is how kubeadm bootstraps the control plane before the API server is ready

■ *Static pods are node-specific — they can't be scheduled across multiple nodes by the scheduler.*

QUESTION 07 · Accenture DevOps Interview

What is the resource hierarchy in GCP?

[GCP] [IAM] [Conceptual]

GCP RESOURCE HIERARCHY (TOP TO BOTTOM)

- **Organization** — Root node. Tied to a Google Workspace/Cloud Identity domain. One per company.
- **Folders** — Optional grouping layer. Organizes by department, team, environment, or BU.
- **Projects** — Primary unit of resource ownership. All GCP resources live in a project.
- **Resources** — Actual GCP services: Compute Engine VMs, GKE clusters, Cloud Storage buckets, etc.

WHY THE HIERARCHY MATTERS

- IAM policies set at a **higher level inherit downward** (Org → Folder → Project → Resource)
- Billing is tracked at the **Project** level
- Quota and API enablement are **per-project**

AWS EQUIVALENT MAPPING

- GCP Organization ≈ AWS Organization (root)
- GCP Folders ≈ AWS OUs (Organizational Units)
- GCP Projects ≈ AWS Accounts
- GCP Resources ≈ AWS Resources

■ *If you come from AWS, this mapping helps bridge conceptual understanding quickly in the interview.*

QUESTION 08 · Accenture DevOps Interview

What is an Identity and IAM Policy in GCP?

[GCP] [IAM] [Security]

IDENTITIES IN GCP

- **Google Account:** Individual user (personal Gmail or Workspace account)
- **Service Account:** Non-human identity for applications/workloads (like AWS IAM Role for EC2)
- **Google Group:** Collection of identities — grant permissions to a group, not individuals
- **Cloud Identity / Workspace Domain:** All users in an org domain

IAM POLICY STRUCTURE

A GCP IAM policy is a set of **bindings**. Each binding = (Role, Members):

```
bindings:
- role: "roles/storage.objectViewer"
members:
- "serviceAccount:my-sa@project.iam.gserviceaccount.com"
- "user:engineer@company.com"
```

ROLE TYPES

- **Primitive Roles:** Owner, Editor, Viewer — broad, avoid in production
- **Predefined Roles:** Fine-grained, service-specific (e.g., roles/container.developer)
- **Custom Roles:** Define exact set of permissions for least-privilege

AWS IAM COMPARISON

- GCP Service Account ≈ AWS IAM Role
- GCP Workload Identity ≈ AWS IRSA (IAM Roles for Service Accounts in EKS)
- GCP IAM Binding ≈ AWS IAM Policy attachment

QUESTION 09 · Accenture DevOps Interview

How do you handle Terraform State?

[Terraform] [State Management] [Critical]

WHAT IS TERRAFORM STATE?

Terraform state (terraform.tfstate) maps your configuration to real infrastructure. It tracks resource IDs, metadata, and dependencies. **Without it, Terraform can't manage existing resources.**

REMOTE STATE — BEST PRACTICE

- **Never store state locally** in production — risk of data loss, no collaboration
- **AWS S3 + DynamoDB:** Standard backend for remote state with locking

```
terraform {
backend "s3" {
bucket = "tf-state-prod"
key = "eks/terraform.tfstate"
region = "ap-south-1"
dynamodb_table = "tf-lock-table"
encrypt = true
}
}
```

STATE LOCKING

- DynamoDB table prevents **concurrent state writes** — avoids corruption
- Lock is acquired on plan/apply, released on completion or timeout

STATE OPERATIONS

- terraform state list — list all tracked resources
- terraform state show — inspect a specific resource
- terraform state mv — rename/move resources without destroy
- terraform state rm — remove from state (resource remains, Terraform forgets it)
- terraform import — bring existing infra under Terraform management

STATE ISOLATION

- Use **separate state files per environment** (dev/staging/prod) — via workspaces or separate backends
- **Terraform Workspaces** allow multiple state files in one backend (simple env separation)

■ *Common interview gotcha: 'What happens if two engineers run apply simultaneously?' → DynamoDB lock prevents it.*

QUESTION 10 · Accenture DevOps Interview

What are your daily routine tasks as a DevOps Engineer?

[DevOps Practice] [Behavioral]

MORNING: OBSERVABILITY & HEALTH CHECK

- Check **Grafana/Datadog dashboards** — CPU, memory, pod restarts, error rates
- Review **Prometheus alerts** fired overnight — triage P1/P2 issues
- Scan **CI/CD pipeline status** — any failed builds or stuck deployments in Jenkins/GitHub Actions

ACTIVE WORKING HOURS: DELIVERY & OPERATIONS

- Review and merge **Terraform PRs** — check plan output, validate no unintended destroys
- Monitor **Argo CD** sync status — ensure GitOps state matches desired config
- Assist developers with **container/K8s issues** — OOMKilled pods, CrashLoopBackOff, image pull errors
- Run **Trivy scans** on new images — flag critical CVEs before prod deploy
- Review **SonarQube reports** — coordinate with dev team on code quality gates

INFRASTRUCTURE & IMPROVEMENTS

- Update **Helm chart values** for app deployments — env-specific configs
- Review **AWS Cost Explorer** — identify idle resources, right-size instances
- Rotate or audit **IAM access keys / secrets** — least-privilege review
- Update **runbooks and documentation** for recurring issues

END OF DAY

- Ensure all **on-call alerts** are acknowledged and handed off
- Close or update tickets in sprint board

■ *Anchor this to your Hisan Labs internship — mention specific tools: Jenkins, Argo CD, Grafana, Datadog, Trivy, SonarQube.*

QUESTION 11 · Accenture DevOps Interview

How do you provision IAM roles or permissions to resources in GCP?

[GCP] [IAM] [Conceptual]

PRIMARY METHOD: IAM BINDINGS VIA GCLOUD / TERRAFORM

- Assign a **predefined or custom role** to an identity (user, group, or service account) at a resource level

```
# gcloud example
gcloud projects add-iam-policy-binding PROJECT_ID \
--member="serviceAccount:my-app@PROJECT.iam.gserviceaccount.com" \
--role="roles/storage.objectAdmin"
```

VIA TERRAFORM (GOOGLE PROVIDER)

```
resource "google_project_iam_member" "app_sa_role" {
project = var.project_id
role = "roles/container.developer"
member = "serviceAccount:${google_service_account.app.email}"
}
```

WORKLOAD IDENTITY (K8S → GCP)

- Bind a **Kubernetes Service Account** to a GCP Service Account
- Eliminates static key files — similar to AWS IRSA pattern
- Grant the GCP SA the roles/iam.workloadIdentityUser role on the K8s SA

BEST PRACTICES

- **Least privilege:** Use predefined roles over primitive roles (Owner/Editor/Viewer)
- **Avoid long-lived keys:** Prefer Workload Identity over Service Account JSON keys
- **Audit:** Use Cloud Audit Logs to track IAM changes

■ *Compare to AWS: this mirrors attaching an IAM role to an EC2/EKS resource via Terraform.*

QUESTION 12 · Accenture DevOps Interview

What are the Kubernetes API Server and Kubelet?

[Kubernetes] [Control Plane] [Core]

KUBE-APISERVER — THE FRONT DOOR

The **API server** is the central control plane component. Every kubectl command, controller, and scheduler communicates exclusively through the API server.

- **Function:** Validates and processes REST API requests, authenticates/authorizes them, persists state to etcd
- **Authentication:** x509 certs, bearer tokens, OIDC, webhook
- **Authorization:** RBAC, ABAC, Node, Webhook modes
- **Admission Controllers:** Mutating + Validating webhooks run after AuthN/AuthZ
- Exposes the K8s REST API — all clients talk HTTPS to :6443

KUBELET — THE NODE AGENT

The **kubelet** runs on every node (control plane + workers). It is responsible for managing pods on its node.

- **Function:** Watches API server for pods scheduled to its node, creates/manages containers via CRI
- **CRI:** Calls container runtime (containerd/CRI-O) to start/stop containers
- **Health Checks:** Runs liveness and readiness probes, reports pod status back to API server
- **Static Pods:** Reads /etc/kubernetes/manifests/ and manages control plane pods
- **Node Registration:** Registers the node with the cluster on startup

FLOW EXAMPLE

- kubectl apply -f pod.yaml → API Server validates + stores in etcd
- Scheduler assigns pod to a node → updates etcd
- Kubelet on that node detects the assignment → pulls image → starts container

■ *Key interview point: kubelet talks to API server, not directly to etcd. etcd is only accessed by the API server.*

QUESTION 13 · Accenture DevOps Interview

What is Bootstrapping in GCP?

[GCP] [GKE] [Infrastructure]

GENERAL DEFINITION

Bootstrapping is the process of setting up infrastructure from scratch — initializing the foundational components before automated tools can take over.

GCP-SPECIFIC CONTEXTS

- **GKE Node Bootstrapping:** When a new node joins a GKE cluster, it runs a startup script to register with the control plane, install kubelet, configure networking (CNI), and mount disks
- **Instance Bootstrap (Compute Engine):** Use startup scripts or cloud-init to configure VM on first boot — install agents, mount storage, configure networking
- **Workload Identity Bootstrap:** Initial setup of service account bindings before workloads can authenticate

TERRAFORM BOOTSTRAPPING

- The **chicken-and-egg problem:** Terraform needs a state bucket — but the bucket must be created before Terraform can manage state
- Solution: create the GCS bucket (or S3 in AWS) **manually or via a bootstrap script** first, then configure backend

```
# Bootstrap sequence
1. gcloud storage buckets create gs://tf-state-bucket
2. Configure terraform backend to use bucket
3. terraform init # now remote state works
```

AWS EQUIVALENT

- EC2 User Data ≈ GCP Startup Scripts
- EKS Managed Node Groups bootstrap ≈ GKE node initialization

■ *If asked about your AWS experience: EC2 user data scripts and EKS node group bootstrap AMIs serve the same purpose.*

QUESTION 14 · Accenture DevOps Interview

Where would you place a database in private vs public subnet, and how would it securely access external traffic?

[AWS] [Networking] [Security] [Architecture]

ANSWER: ALWAYS PRIVATE SUBNET

Databases should **never** be placed in a public subnet. Exposing DB ports (5432, 3306, 27017) to the internet is a critical security risk.

CORRECT ARCHITECTURE

- **Database placement:** Private subnet only — no public IP, no IGW route
- **Application tier:** EC2/EKS in private subnet → connect to RDS via private IP within VPC
- **Security Group:** RDS SG allows inbound only from app server SG (not 0.0.0.0/0)
- **Subnet Group:** RDS Multi-AZ — place in private subnets across at least 2 AZs

HOW THE DATABASE ACCESSES EXTERNAL TRAFFIC (OUTBOUND)

- **Option 1 — NAT Gateway:** Route outbound traffic from private subnet through NAT GW in public subnet
  - Use case: RDS calling external APIs, pulling updates
- **Option 2 — VPC Endpoints:** Preferred for AWS services — private connectivity without internet
  - S3 Gateway Endpoint — RDS Proxy or Lambda accessing S3
  - Secrets Manager Interface Endpoint — DB credentials rotation without NAT
- **Option 3 — PrivateLink:** Private connectivity to SaaS or cross-account services

SECURE ACCESS PATTERN SUMMARY

- Internet → ALB (public subnet) → EC2/EKS App (private) → RDS (private)
- No part of this flow exposes the database IP to the public internet
- Credentials stored in **AWS Secrets Manager**, rotated automatically — not hardcoded

■ *Bonus point: mention RDS Proxy for connection pooling + IAM authentication — shows production maturity.*